

The 8085 Master Manual

Volume V: Commanding the Machine
(Assembly Language Programming)

A Grassroots Engineering Guide

Transforming raw silicon potential into algorithmic execution.

*"Hardware dictates what is possible; Software dictates what is
actualized."*

Chapter 1: The Lexicon of Silicon

1.1 The Bridge Between Mind and Machine

In Volumes I through IV, we constructed the physical machine. We established the 16-bit address bus, the 8-bit data bus, the internal registers, the ALU, the timing sequencers, and the decoding logic. However, the machine currently sits in complete silence. It possesses the capability to process information, but it lacks instructions.

The microprocessor only understands **Machine Language**—patterns of 1s and 0s (e.g., `0100 0111`). Writing programs purely in binary is error-prone and unintuitive for human engineers. To bridge this gap, Intel developed **Assembly Language**.

Assembly Language uses short, English-like acronyms called **Mnemonics** to represent machine code instructions. For example, instead of writing `0100 0111`, we write `MOV B, A`. A program called an *Assembler* is then used to translate these mnemonics back into the binary opcodes the silicon can decode.

1.2 The Anatomy of an Instruction

Every instruction in the 8085 consists of two fundamental components:

1. **The Opcode (Operation Code):** What specific action is to be performed (e.g., Move, Add, Subtract, Jump).
2. **The Operand:** The data upon which the operation is performed, or the address where the data resides.

Instructions can vary in length depending on how much information is required to define the operand:

- **1-Byte Instructions:** The operand is implicit or located in an internal register (e.g., `MOV B, A` or `CMA`).
- **2-Byte Instructions:** The first byte is the opcode, and the second byte is an 8-bit piece of data or an 8-bit I/O port address (e.g., `MVI A, 05H`).
- **3-Byte Instructions:** The first byte is the opcode, and the subsequent two bytes form a 16-bit memory address (e.g., `LDA 8000H`).

Chapter 2: Addressing Modes

A microprocessor is ultimately a data manipulation engine. Therefore, the method by which a programmer provides data to an instruction is of paramount importance. The 8085 architecture defines five distinct **Addressing Modes** to locate operands.

2.1 Register Addressing

In this mode, the data to be operated upon is already stored in one of the microprocessor's internal registers (A, B, C, D, E, H, or L). Because the data does not need to be fetched from external memory, these instructions are executed extremely rapidly (usually in a single 4T Opcode Fetch cycle).

EXAMPLE:

MOV B, A

MEANING:

Copy the 8-bit contents of the Accumulator (Register A) into Register B.
The contents of A remain unchanged.

EXAMPLE:

ADD C

MEANING:

Add the contents of Register C to the Accumulator.

2.2 Immediate Addressing

In this mode, the actual 8-bit or 16-bit data is provided *immediately* following the opcode within the instruction itself. The letter '**I**' in a mnemonic is universally used by Intel to denote Immediate addressing.

EXAMPLE:

MVI A, 3FH

MEANING:

Move the 8-bit immediate data **3FH** directly into the Accumulator. This is a 2-byte instruction.

EXAMPLE:

LXI H, 2050H

MEANING:

Load the 16-bit immediate data **2050H** directly into the Register Pair HL. The high-byte (20H) goes to H, and the low-byte (50H) goes to L. This is a 3-byte instruction.

2.3 Direct Addressing

In this mode, the instruction explicitly states the exact 16-bit memory address (or 8-bit I/O port address) where the data resides.

EXAMPLE:

STA 8050H

MEANING:

Store the contents of the Accumulator Directly into memory address 8050H.

EXAMPLE:

IN 01H

MEANING:

Read an 8-bit byte Directly from I/O Port 01H into the Accumulator.

2.4 Register Indirect Addressing

This is the most powerful addressing mode for algorithmic programming, particularly when dealing with arrays. The instruction does not contain the memory address. Instead, it points to an internal register pair (usually HL) that currently holds the 16-bit memory address.

In the 8085 assembler, the operand '**M**' (Memory) is exclusively used to denote Register Indirect addressing via the HL pair.

EXAMPLE:

MOV A, M

MEANING:

The CPU looks at the HL register pair. Suppose HL contains **8000H**. The CPU then goes to external memory address **8000H**, reads the 8-bit data stored there, and moves it into the Accumulator.

WHY IT IS POWERFUL:

If you want to read the next byte in an array, you do not need to write a new instruction with a new direct address. You simply use the 1-byte instruction **INX H** (Increment HL pair) followed by another **MOV A, M**.

2.5 Implicit (Implied) Addressing

In this mode, the operand is hidden within the design of the opcode itself. There is no external data and no register specified. The instruction operates on a fixed internal architecture, usually the Accumulator or the Flag Register.

EXAMPLE:

CMA (Complement Accumulator)

MEANING:

Invert all the bits currently residing in the Accumulator (1s become 0s, 0s become 1s). The operand (the Accumulator) is implied.

EXAMPLE:

STC (Set Carry)

MEANING:

Force the Carry Flag in the Flag Register to a logic 1.

Chapter 3: Control Flow and Looping

A microprocessor that only executes instructions linearly from top to bottom is merely a calculator. The true power of computation lies in **Conditional Branching**—the ability to make decisions and loop back based on the results of previous calculations.

3.1 The Mechanics of the Jump

The 8085 controls execution sequence using the Program Counter (PC). To branch to a different part of the code, the CPU must overwrite the contents of the PC with a new 16-bit target address.

Unconditional Jump: The `JMP <16-bit address>` instruction forces the PC to load the new address regardless of the current processor state.

3.2 Conditional Jumps and the Flag Register

Conditional jumps allow the CPU to ask questions about the Flag Register before deciding whether to jump or simply continue to the next sequential instruction.

Instruction	Condition to Jump	Flag Monitored	Common Use Case
JZ (Jump on Zero)	Zero Flag (Z) = 1	Zero	Checking if two numbers are equal (after a CMP), or if a counter has reached 0.
JNZ (Jump on Not Zero)	Zero Flag (Z) = 0	Zero	Looping back repeatedly until a counter register decrements to exactly zero.
JC (Jump on Carry)	Carry Flag (CY) = 1	Carry	Checking for mathematical overflow, or determining if Accumulator < Register during a CMP.
JNC (Jump on No Carry)	Carry Flag (CY) = 0	Carry	Determining if Accumulator ≥ Register during a CMP.

3.3 The Architecture of a Loop

Almost all array processing, multiplication, and software delay generation relies on a standard looping architecture using a General Purpose Register as a counter.

THE ANATOMY OF A SOFTWARE LOOP

```
        MVI C, 05H      ; Step 1: Initialize the Counter (Load
C with 5)
LOOP:   ...             ; Step 2: Perform the intended action
        ...             ; (e.g., adding numbers, moving data)
        DCR C           ; Step 3: Decrement the Counter (C = C
- 1)
        JNZ LOOP        ; Step 4: Check Zero Flag. If Z=0 (C is
not 0), Jump back to LOOP.
        HLT             ; Step 5: When C hits 0, JNZ fails, and
execution escapes the loop.
```

Notice how **DCR C** is critical. Unlike **DCX H** (which decrements a 16-bit pair but does *not* affect flags), the 8-bit **DCR** instruction updates the Zero Flag. When C drops from 1 to 0, the ALU sets the Zero Flag to 1. The subsequent **JNZ** instruction sees **Z=1** and safely terminates the loop.

Chapter 4: The Subroutine and The Stack

4.1 The Concept of Modularity

In advanced programming, certain functional blocks (like generating a 1-second time delay, or multiplying two numbers) are required repeatedly throughout a main program. Writing the same loop structure ten times consumes excessive ROM space and makes the code illegible.

The **Subroutine** solves this by allowing a block of code to be stored once in memory and invoked from anywhere.

4.2 The Mechanics of CALL and RET

When the CPU encounters a **CALL** <16-bit address> instruction, it cannot simply overwrite the Program Counter (like a **JMP** instruction). If it did, it would have no idea how to return to the main program once the subroutine finished.

It must utilize the **Stack** (managed by the Stack Pointer, SP) to memorize its return path.

THE SUBROUTINE EXECUTION SEQUENCE

1. **FETCH:**

The CPU fetches the 3-byte **CALL** instruction. As it fetches, the Program Counter (PC) automatically increments and now points to the *next* instruction in the main program. This is the

RETURN ADDRESS

.

2. **SAVE STATE (PUSH):**

The CPU automatically decrements the Stack Pointer (SP) and pushes the high-order byte of the PC onto the stack. It decrements the SP again and pushes the low-order byte. The Return Address is now safely stored in external RAM.

3. **JUMP:**

The CPU loads the target subroutine address into the PC. Execution shifts to the subroutine.

4. **EXECUTE SUBROUTINE:**

The subroutine code runs.

5. **RETURN (POP):**

The very last instruction of every subroutine must be **RET**. The **RET** instruction instructs the CPU to pop the top two bytes off the stack and load them back into the Program Counter. The CPU instantly "wakes up" exactly where it left off in the main program.

4.3 Stack Initialization

Because the Stack resides in external Read/Write memory (RAM), the CPU does not inherently know where the Stack is located. If a programmer uses **CALL** or **PUSH** without initializing the Stack Pointer, the CPU will write data to a random memory location, potentially overwriting the program itself!

Therefore, *every* program that uses subroutines or interrupts must begin with the instruction:

LXI SP, <Top_Of_RAM_Address> (e.g., **LXI SP, FFFFH**).

Chapter 5: Delay Mathematics

Because the 8085 operates synchronously with an external crystal oscillator, every instruction consumes a mathematically exact amount of time (measured in T-states). By forcing the processor into a useless, repetitive counting loop, we can generate precise hardware time delays without using external timer chips (like the 8253).

5.1 The MH Timing Methodology

To calculate a software delay, we must sum the T-states of the loop instructions and multiply by the number of iterations.

However, Professor MH's methodology requires a critical structural correction regarding the **JNZ** instruction:

- When the jump condition is **True** (the counter is not zero, and the CPU jumps), the **JNZ** instruction requires **10 T-states** (to fetch the target address and overwrite the PC).
- When the jump condition is **False** (the counter hits zero, and the CPU escapes the loop), the **JNZ** instruction requires only **7 T-states** (it fetches the target address, but aborts the PC overwrite).

Therefore, the strict formula for an 8-bit loop delay is:

$$\text{Total T-States} = T_{\{\text{setup}\}} + (T_{\{\text{loop}\}} \times N) - 3$$

Where N is the decimal loop count, and the "- 3" accounts for the final failed jump ($10 - 7 = 3$).

5.2 Nested Loop Calculation Example

EXAMPLE: A 16-BIT NESTED DELAY

```

        LXI B, 0FFFH      ; Setup: 10 T-states. (0FFFH = 4095
Decimal)
LOOP:   DCX B              ; Loop: 6 T-states
        MOV A, C          ; Loop: 4 T-states (Moving C to check
if BC is zero)
        ORA B             ; Loop: 4 T-states (Logical OR sets Z
flag if B and C are 0)
        JNZ LOOP         ; Loop: 10 T-states (7 on last
iteration)

```

STEP 1: DETERMINE LOOP PARAMETERS

$$T_{\text{setup}} = 10$$

$$T_{\text{loop}} = 6 + 4 + 4 + 10 = 24 \text{ T-states per iteration.}$$

$$N = 4095 \text{ Iterations.}$$

STEP 2: APPLY FORMULA

$$\text{Total } T = 10 + (24 \text{ times } 4095) - 3$$

$$\text{Total } T = 10 + 98280 - 3 = 98287 \text{ T-states.}$$

STEP 3: CONVERT TO SECONDS

Assume a 2 MHz system clock ($1\ T_{\text{ext}} = 0.5\ \mu\text{s}$).

Time = $98287 \times 0.5\ \mu\text{s} = 49143.5\ \mu\text{s} =$

0.049 SECONDS (APPROX 50 MS)

.

Chapter 6: Algorithmic Synthesis

(Mastering the Logic)

We have established the vocabulary (Instructions), the grammar (Addressing Modes), and the sentence structure (Loops and Subroutines). We must now synthesize these elements to construct full-scale engineering algorithms.

The following modules dissect the foundational programs required by the syllabus, explaining the logical architecture behind the code.

6.1 Mathematical Limitations: Multiplication

The 8085 ALU contains adders and subtractors, but it lacks a dedicated hardware multiplier. Therefore, multiplication must be derived logically through **Repeated Addition**. To calculate $A \times B$, we add the value of A to an accumulator exactly B times.

ALGORITHM: 8-BIT MULTIPLICATION

Objective: Multiply data at **8000H** by data at **8001H**. Store the product at **8100H**.

```

; Initialization Phase
        LDA 8000H      ; Load the Multiplicand into
Accumulator
        MOV B, A       ; Safely store Multiplicand in Register
B
        LDA 8001H      ; Load the Multiplier into Accumulator
        MOV C, A       ; Store Multiplier in Register C (Acts
as Loop Counter)
        MVI A, 00H     ; Clear the Accumulator. It will hold
the Running Product.

; Repeated Addition Loop
MLOOP:  ADD B          ; Add the Multiplicand to the Running
Product in A
        DCR C          ; Decrement the Multiplier Counter
        JNZ MLOOP      ; If Counter is not zero, loop back and
add again

; Storage Phase
        STA 8100H      ; Store the final Product in memory
        HLT           ; Terminate Program

```

LOGICAL VULNERABILITY:

What happens if the product of the two 8-bit numbers exceeds 255 (FFH)? The 8-bit Accumulator will overflow, setting the Carry Flag, but the upper bits of the true 16-bit product will be lost. Advanced multiplication algorithms must use a 16-bit register pair (like HL) to track the carry overflows to generate a true 16-bit product.

6.2 The BCD Anomaly and The DAA Instruction

Humans operate in Base-10 (Decimal). Microprocessors operate in Base-2 (Binary).

To interface with human-readable displays (like 7-segment LEDs), engineers use **Binary Coded Decimal (BCD)**. In BCD, a byte is split into two 4-bit nibbles. Each nibble is restricted to representing a decimal digit from 0 to 9 (**0000** to **1001**). The hex values A, B, C, D, E, and F (10 through 15) are strictly illegal in BCD.

The Arithmetic Failure

If we add BCD **38** and BCD **45** in the 8085, the ALU performs standard binary hex addition:

$$38H + 45H = 7DH.$$

The result **7DH** contains the illegal digit 'D'. The human display will output garbage. We need the answer to be BCD **83**.

Intel solved this by inventing the **DAA (Decimal Adjust Accumulator)** instruction. It must be executed immediately after any **ADD**, **ADC**, or **ADI** instruction.

The Hardware Correction Logic of DAA:

- If the lower nibble is greater than 9, OR if the Auxiliary Carry (AC) flag was set during the addition, the ALU automatically adds **06H** to the Accumulator.

- If the upper nibble is greater than 9, OR if the Carry (CY) flag was set during the addition, the ALU automatically adds **60H** to the Accumulator.

EXECUTING THE CORRECTION:

ALU Result: 7DH.

The lower nibble 'D' (13) is greater than 9. DAA triggers rule #1 and adds 06H.

$7DH + 06H = 83H$.

The Accumulator now holds **83H**, which is the mathematically perfect BCD representation of $38 + 45$.

ALGORITHM: BCD ADDITION

```

        LXI H, 8000H    ; Point to first BCD number
        MOV A, M        ; Load first number into Accumulator
        INX H           ; Point to second BCD number
        ADD M           ; Binary addition of the two numbers
        DAA             ; MAGIC: Instantly correct the binary
sum to valid BCD
        STA 8100H       ; Store the correct BCD result
        HLT

```

6.3 Array Processing: Finding the Maximum Value

Processing sequential arrays of data is the primary justification for **Register Indirect Addressing**. The HL register pair acts as an infinitely sliding pointer.

ALGORITHM: LARGEST NUMBER IN AN ARRAY

Objective: Address **8000H** holds the length of the array (N). The data starts at **8001H**. Find the maximum value and store it at **9000H**.

Logic: We will use the Accumulator as the "Current Champion" register. We load the first element into the Accumulator. We then fetch the next element and compare it (**CMP**) against the Accumulator. If the new element is larger, it becomes the new Champion.

```

        LXI H, 8000H      ; HL points to the array length N
        MOV C, M          ; Load the length N into the loop
counter C
        DCR C            ; Decrement C (Because we will consume
the 1st element directly)

        INX H            ; HL now points to the first array
element
        MOV A, M          ; Load 1st element into Accumulator
(The initial Champion)

LOOP:   INX H            ; Point HL to the NEXT element in the
array
        CMP M            ; Compare Champion (A) with Challenger
(M)
                                ; CMP internally subtracts M from A (A
- M)

        JNC NEXT         ; Jump if No Carry. If A >= M, no
borrow is needed.
                                ; The Champion is still the largest.
Skip to NEXT.

        MOV A, M          ; If Carry is generated, A < M. The
Challenger won!
                                ; Move the new larger value from Memory
into Accumulator.

NEXT:   DCR C            ; Decrement array counter
        JNZ LOOP         ; If not zero, go back and check the
next element

```

```
        STA 9000H        ; The loop is finished. Store the  
ultimate Champion.  
        HLT
```

INSTRUCTION INSIGHT:

The **CMP M** instruction is elegant because it alters the Flag Register (specifically the Carry Flag) without destroying the data sitting in the Accumulator or the Memory. It is a purely diagnostic subtraction.

6.4 The Fibonacci Sequence Generation

The Fibonacci sequence (0, 1, 1, 2, 3, 5, 8...) is governed by the recurrence relation $F_n = F_{n-1} + F_{n-2}$. Translating this into silicon requires sliding window memory management. We must constantly shift our variables so that the "current sum" becomes the "previous number" for the next iteration.

ALGORITHM: FIBONACCI GENERATOR

Objective: Address **8000H** dictates how many terms (N) to generate.
Store the sequence starting at **8001H**.

```

        LXI H, 8000H    ; HL points to N
        MOV C, M        ; Load N into counter C
        INX H           ; Move pointer to 8001H for the first
output
; Initialize Term 1 (0)
        MVI D, 00H      ; Register D acts as F(n-2). Set to 0.
        MOV M, D        ; Store 0 in memory
        INX H           ; Move memory pointer forward
        DCR C           ; Decrement total term counter
        JZ END          ; Edge case: If user only asked for 1
term, stop.

; Initialize Term 2 (1)
        MVI E, 01H      ; Register E acts as F(n-1). Set to 1.
        MOV M, E        ; Store 1 in memory
        INX H           ; Move memory pointer forward
        DCR C           ; Decrement total term counter
        JZ END          ; Edge case: If user only asked for 2
terms, stop.

; Main Generation Loop
LOOP:   MOV A, D         ; Load F(n-2) into Accumulator
        ADD E           ; Add F(n-1) to Accumulator. A now
holds the new F(n).
        MOV M, A        ; Store the newly calculated F(n) in
memory

; The Sliding Window Shift
        MOV D, E        ; The old F(n-1) shifts left to become
the new F(n-2)

```

```
        MOV E, A          ; The newly calculated sum shifts to
become the new F(n-1)

        INX H             ; Advance memory pointer for the next
iteration

        DCR C             ; Decrement total term counter

        JNZ LOOP          ; Loop until N terms are generated

END:    HLT               ; Terminate Program
```

With the mastery of these algorithmic architectures, the microprocessor transitions from a static hardware construct into a dynamic computational engine. We have traversed the entire vertical stack—from the voltage on a physical pin (Volume I), to the internal registers (Volume II), the timing pulses (Volume III), the memory map (Volume IV), and finally, the software logic (Volume V).

The Intel 8085 machine has been fully deconstructed and rebuilt in your mind. You are now prepared to command the silicon.

END OF VOLUME V

End of the Master Manual